

DEVELOPERS HUB CORPORATION

Cybersecurity Internship Program

3 WEEKS REPORT

Security Assessment | Implementation | Penetration Testing

WEEK 1

Security Assessment

WEEK 2

Security Implementation

WEEK 3

Penetration Testing & HTTPS

Prepared By:

Ayesha Sarwar Khan

Organization:

Developers Hub Corporation

Internship:

Cybersecurity Internship

Submission Date:

June 26, 2026

Duration:

June 10 – June 12, 2026

Application:

DVWA & vulnerable-node (Node.js/Express)

TABLE OF CONTENTS

WEEK 1 — Security Assessment & Vulnerability Analysis

- 1.1 Executive Summary
- 1.2 Scope & Methodology
- 1.3 Tools Used
- 1.4 OWASP ZAP Automated Scan
- 1.5 Detailed Vulnerability Findings
- 1.6 Findings Summary Table
- 1.7 Recommendations
- 1.8 Conclusion

WEEK 2 — Security Implementation & Vulnerability Remediation

- 2.1 Executive Summary
- 2.2 Application Under Test
- 2.3 Pre-Fix Vulnerabilities
- 2.4 Fix 1 — Secure HTTP Headers (Helmet.js)
- 2.5 Fix 2 — Input Validation (validator)
- 2.6 Fix 3 — Password Hashing (bcrypt)
- 2.7 Fix 4 — SQL Injection Prevention
- 2.8 Fix 5 — JWT Token-Based Authentication
- 2.9 Before vs After Comparison
- 2.10 Files Modified & Packages Installed
- 2.11 Conclusion

WEEK 3 — Advanced Security & Final Reporting

- 3.1 Executive Summary
 - 3.2 Basic Penetration Testing — Nmap
 - 3.3 Basic Logging Implementation — Winston
 - 3.4 Secure Data Transmission — HTTPS
 - 3.5 Final Security Best Practices Checklist
 - 3.6 Conclusion
-

WEEK 1 — SECURITY ASSESSMENT & VULNERABILITY ANALYSIS

Application: DVWA (Damn Vulnerable Web Application) | Tools: OWASP ZAP, Browser DevTools, Docker | Date: June 10, 2026

Prepared By	Ayesha Sarwar Khan
Organization	Developers Hub Corporation
Date	June 10, 2026
Application Tested	DVWA (Damn Vulnerable Web Application)
Assessment Type	Web Application Security Assessment
Tools Used	OWASP ZAP, Browser DevTools, Docker

1.1 Executive Summary

This report presents the findings of a comprehensive security assessment conducted on DVWA (Damn Vulnerable Web Application), a deliberately vulnerable web application used for cybersecurity training. The assessment was performed as part of Week 1 of the Cybersecurity Internship at Developers Hub Corporation. The assessment combined automated scanning using OWASP ZAP with manual penetration testing. A total of 10 vulnerabilities were identified, including 3 critical high-risk vulnerabilities discovered through manual testing.

Risk Level	Count	Detection Method
HIGH	3	Manual Testing
MEDIUM	2	OWASP ZAP
LOW	5	OWASP ZAP
Informational	2	OWASP ZAP
TOTAL	12	—

1.2 Scope & Methodology

Application Under Test

The target was DVWA running locally via Docker on <http://localhost:3000>. DVWA is an intentionally vulnerable PHP/MySQL web application designed for cybersecurity training. It provides a safe and legal environment to practice common web application attack techniques.

Application Pages Explored

The following pages and vulnerability modules were explored during the assessment: Home/Dashboard, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, and DVWA Security settings.

1.3 Tools Used

Tool	Purpose
DVWA via Docker	Target mock web application for testing
OWASP ZAP	Automated vulnerability scanner
Browser DevTools	Manual XSS & SQL injection testing
Docker Desktop	Container to run DVWA locally

1.4 OWASP ZAP Automated Scan

An automated scan was performed against <http://localhost:3000> using OWASP ZAP. The scan identified 9 alerts across medium, low, and informational risk levels. The scanner crawled all accessible pages and tested each for common web application vulnerabilities using passive and active scanning rules.

ZAP Alerts Identified:

#	Alert	Risk Level
1	Content Security Policy (CSP) Header Not Set	MEDIUM
2	Missing Anti-Clickjacking Header	MEDIUM
3	Cookie No HttpOnly Flag	LOW
4	Cookie Without SameSite Attribute	LOW
5	In Page Banner Information Leak	LOW
6	Server Leaks Version Information via Server Header	LOW
7	X-Content-Type-Options Header Missing	LOW
8	Authentication Request Identified	Informational

#	Alert	Risk Level
9	Session Management Response Identified	Informational

1.5 Detailed Vulnerability Findings

Finding 1: Reflected Cross-Site Scripting (XSS)

Attribute	Detail
Risk Level	HIGH — Manually Confirmed
Location	XSS (Reflected) Page — Name input field
Payload Used	alert('XSS')
Result	Alert popup appeared — vulnerability confirmed

Description: Reflected XSS occurs when malicious scripts injected into input fields are immediately reflected in the server response and executed by the browser. User input was not sanitized before being rendered on the page. An attacker could craft a malicious URL and trick a victim into clicking it, causing arbitrary JavaScript to execute in their browser session — enabling session hijacking, credential theft, or page defacement.

Finding 2: Stored Cross-Site Scripting (XSS)

Attribute	Detail
Risk Level	HIGH — Manually Confirmed
Location	XSS (Stored) Page — Guestbook Message field
Payload Used	alert('Stored XSS')
Result	Script stored in DB — alert fires for every visitor

Description: Stored XSS is more dangerous than Reflected XSS because the malicious script is permanently saved in the database and executes for every user who visits the affected page. The guestbook message field accepted and stored raw JavaScript without any sanitization. Every subsequent visitor to the guestbook page triggered the alert, confirming persistent execution.

Finding 3: SQL Injection

Attribute	Detail
Risk Level	HIGH — Manually Confirmed
Location	SQL Injection Page — User ID field & Login page

Attribute	Detail
Payload Used	1' OR '1'='1 / admin' OR '1'='1
Result	All database records exposed — authentication bypassed

Description: SQL Injection occurs when unsanitized user input is included directly in database queries. The crafted payload manipulated the SQL query logic to return all user records, exposing sensitive data including usernames and password hashes. The login authentication was also successfully bypassed using the payload admin' OR '1'='1, granting unauthorized access without knowing the correct password.

Finding 4: Content Security Policy (CSP) Header Not Set

Attribute	Detail
Risk Level	MEDIUM
Detection	OWASP ZAP Automated Scan

Description: The application does not define a Content-Security-Policy header, making XSS attacks easier since the browser has no policy restricting which scripts can run. Without CSP, any injected script executes freely with no browser-level enforcement.

Finding 5: Missing Anti-Clickjacking Header

Attribute	Detail
Risk Level	MEDIUM
Detection	OWASP ZAP Automated Scan

Description: The application lacks X-Frame-Options or CSP frame-ancestors headers, allowing it to be embedded in malicious iframes on attacker-controlled pages, enabling clickjacking attacks where victims are tricked into clicking hidden UI elements.

Finding 6: Cookie No HttpOnly Flag

Attribute	Detail
Risk Level	LOW
Detection	OWASP ZAP Automated Scan

Description: Session cookies are not marked with the HttpOnly flag, meaning they can be accessed via JavaScript, making cookie theft via XSS significantly easier. An attacker exploiting XSS could steal the session cookie and hijack the user's authenticated session.

Finding 7: Cookie Without SameSite Attribute

Attribute	Detail
Risk Level	LOW
Detection	OWASP ZAP Automated Scan

Description: Cookies lack the SameSite attribute making them vulnerable to Cross-Site Request Forgery (CSRF) attacks. Without SameSite=Strict or Lax, cookies are sent with cross-site requests enabling forged actions on behalf of authenticated users.

Finding 8: In Page Banner Information Leak

Attribute	Detail
Risk Level	LOW
Detection	OWASP ZAP Automated Scan

Description: The application reveals server version information (Apache/2.4.25) in page content, giving attackers useful targeting information to identify known exploits for that specific version.

Finding 9: Server Leaks Version Information

Attribute	Detail
Risk Level	LOW
Detection	OWASP ZAP Automated Scan

Description: HTTP response headers expose the web server software and version (Apache/2.4.25 Debian), helping attackers identify known CVEs and exploits targeting this exact software version.

Finding 10: X-Content-Type-Options Header Missing

Attribute	Detail
Risk Level	LOW
Detection	OWASP ZAP Automated Scan

Description: The X-Content-Type-Options header is absent, allowing browsers to MIME-sniff responses which can lead to unintended script execution by misinterpreting content types.

Findings 11 & 12: Informational Alerts

Authentication Request Identified & Session Management Response Identified — These are informational alerts only, not vulnerabilities. OWASP ZAP detected the login form and session cookie, confirming it successfully identified the application's authentication mechanism for further targeted testing.

1.6 Findings Summary Table

#	Vulnerability	Risk	Detection
1	Reflected Cross-Site Scripting (XSS)	HIGH	Manual
2	Stored Cross-Site Scripting (XSS)	HIGH	Manual
3	SQL Injection	HIGH	Manual
4	CSP Header Not Set	MEDIUM	OWASP ZAP
5	Missing Anti-Clickjacking Header	MEDIUM	OWASP ZAP
6	Cookie No HttpOnly Flag	LOW	OWASP ZAP
7	Cookie Without SameSite Attribute	LOW	OWASP ZAP
8	In Page Banner Information Leak	LOW	OWASP ZAP
9	Server Leaks Version Information	LOW	OWASP ZAP
10	X-Content-Type-Options Header Missing	LOW	OWASP ZAP

1.7 Areas of Improvement & Recommendations

#	Vulnerability	Recommendation	Priority
1	Reflected & Stored XSS	Sanitize all user inputs. Implement output encoding. Add strict CSP headers.	CRITICAL
2	SQL Injection	Use parameterized queries. Never concatenate user input into SQL. Apply least-privilege to DB accounts.	CRITICAL
3	Missing CSP Header	Implement a strict Content-Security-Policy header on all pages.	HIGH
4	Anti-Clickjacking	Add X-Frame-Options: DENY or use CSP frame-ancestors directive.	HIGH
5	Cookie Flags	Set HttpOnly, Secure, and SameSite=Strict on all session cookies.	MEDIUM

#	Vulnerability	Recommendation	Priority
6	Info Disclosure	Suppress server version info from HTTP headers and page banners.	MEDIUM
7	X-Content-Type	Add X-Content-Type-Options: nosniff header to all responses.	LOW

1.8 Conclusion

This security assessment of DVWA successfully identified 10 vulnerabilities across multiple risk levels. The three most critical findings — Reflected XSS, Stored XSS, and SQL Injection — were confirmed through manual penetration testing and represent severe real-world risks including session hijacking, data theft, and unauthorized database access. Notably, all three high-risk vulnerabilities were discovered through manual testing rather than automated scanning, demonstrating the importance of combining both approaches for a thorough assessment. The OWASP ZAP automated scan identified 7 additional security misconfigurations that collectively weaken the application's security posture. The findings from this assessment directly informed the remediation work carried out in Week 2.

WEEK 2 — SECURITY IMPLEMENTATION & VULNERABILITY REMEDIATION

Application: vulnerable-node (NodeVulnerable) — Node.js/Express | Tools: bcrypt, jsonwebtoken, helmet, validator, PostgreSQL | Date: June 11, 2026

Prepared By	Ayesha Sarwar Khan
Organization	Developers Hub Corporation
Date	June 11, 2026
Application	NodeVulnerable (vulnerable-node) — Node.js/Express
Assessment Type	Security Implementation & Code Remediation
Tools Used	Node.js, bcrypt, jsonwebtoken, helmet, validator, PostgreSQL

2.1 Executive Summary

This report documents the security improvements implemented during Week 2 of the Cybersecurity Internship at Developers Hub Corporation. Building directly on the vulnerabilities identified in the Week 1 assessment of DVWA, this week's work involved applying practical security fixes to a Node.js/Express web application (vulnerable-node) — a mock e-commerce User Management System. Four categories of security fixes were implemented, addressing the critical vulnerabilities found in Week 1 including XSS, SQL Injection, weak authentication, and missing security headers. All fixes were successfully deployed and verified on a local PostgreSQL-backed instance of the application.

Metric	Value
Vulnerability Categories Fixed	4
Security Libraries Added	5
Files Modified	3
All Fixes Verified	Yes

Fix	Vulnerability Addressed	Library	Status
Secure HTTP Headers	Missing CSP, X-Frame-Options, XSS headers	helmet@4	Implemented
Input Validation	XSS, empty input attacks	validator	Implemented

Fix	Vulnerability Addressed	Library	Status
Password Hashing	Plaintext password storage	bcrypt	Implemented
SQL Injection Prevention	SQL Injection in all DB queries	pg (parameterized)	Implemented
JWT Authentication	Weak/no token-based auth	jsonwebtoken	Implemented

2.2 Application Under Test

Property	Details
Application	vulnerable-node (NodeVulnerable)
Framework	Node.js + Express
Database	PostgreSQL 15 (local)
URL	http://localhost:3000
Pages	Login, Products, Product Detail, Search, Purchase History
Setup	npm install → set STAGE=LOCAL → npm start

2.3 Pre-Fix Vulnerabilities Identified

Before implementing fixes, the following vulnerabilities were present in the application, directly mirroring the findings from the Week 1 DVWA assessment:

#	Vulnerability	Risk	Location in Code
1	Missing Security Headers (CSP, X-Frame-Options, etc.)	MEDIUM	app.js — no headers middleware
2	No Input Validation	HIGH	routes/login.js — raw req.body usage
3	Plaintext Password Storage	HIGH	model/auth.js — direct string comparison
4	SQL Injection	HIGH	model/auth.js, model/products.js — string concatenation
5	No Token-Based Authentication	MEDIUM	routes/login.js — session only, no JWT

2.4 Fix 1 — Secure HTTP Headers (Helmet.js)

Vulnerability Addressed

The application had no security-related HTTP response headers set. This allowed XSS attacks to run freely (no CSP), enabled clickjacking (no X-Frame-Options), and allowed MIME sniffing attacks (no X-Content-Type-Options) — all issues identified in the Week 1 ZAP scan.

Fix Applied

Installed helmet@4 and added it as Express middleware in app.js:

```
// app.js – added after express initialization
var helmet = require('helmet');
var app = express();
app.use(helmet()); // Adds all security headers automatically
```

Security Headers Applied After Fix:

Header	Value
Content-Security-Policy	default-src 'self'; object-src 'none'; script-src 'self'...
X-Frame-Options	SAMEORIGIN
Strict-Transport-Security	max-age=15552000; includeSubDomains
X-Content-Type-Options	nosniff
X-XSS-Protection	0 (modern CSP-based protection active)
Referrer-Policy	no-referrer
X-DNS-Prefetch-Control	off
X-Download-Options	noopen
X-Permitted-Cross-Domain-Policies	none

Verification: Opening Chrome DevTools → Network tab → login request → Response Headers confirmed the Content-Security-Policy header is present. The browser console also showed: "Executing inline script violates the Content Security Policy directive 'script-src self'" — confirming Helmet's CSP is actively blocking unsafe inline scripts.

2.5 Fix 2 — Input Validation (validator library)

Vulnerability Addressed

User inputs (username, password) were taken directly from req.body without any validation. An attacker could submit empty strings, excessively long strings, or scripts as inputs.

Before (Vulnerable) — routes/login.js:

```
var user = req.body.username; // No validation
var password = req.body.password; // No validation
```

```
auth(user, password) // Passed directly to DB
```

Fix Applied — routes/login.js:

```
var validator = require('validator');

if (!user || validator.isEmpty(user.trim())) {
  return res.redirect("/login?error=Username is required");
}

if (!password || validator.isEmpty(password.trim())) {
  return res.redirect("/login?error=Password is required");
}

if (!validator.isLength(user, { min: 3, max: 50 })) {
  return res.redirect("/login?error=Invalid username length");
}

user = validator.escape(user); // Converts < > " to safe HTML entities
```

Verification: Submitting an empty username on the login form displayed a validation error message, confirming inputs are now being validated before reaching the server. The `validator.escape()` function converts dangerous characters to HTML entities, blocking XSS payloads.

2.6 Fix 3 — Password Hashing (bcrypt)

Vulnerability Addressed

Passwords were stored as plaintext in the PostgreSQL database and compared directly as strings. A database breach would instantly expose all user passwords. This mirrors the weak password storage vulnerability identified in Week 1.

Before (Vulnerable) — model/auth.js:

```
// Passwords stored as plaintext in DB:
// admin | admin
// roberto | asdfpiuw981
var q = "SELECT * FROM users WHERE name = '" + username +
  "' AND password ='" + password + "';";
// Plaintext comparison – completely insecure
```

Fix Applied — model/auth.js:

```
var bcrypt = require('bcrypt');

return client.query("SELECT * FROM users WHERE name = $1;", [username])
  .then(function(result) {
    var user = result.rows[0];
```

```

return bcrypt.compare(password, user.password)
.then(function(match) {
if (!match) { throw new Error("Invalid username or password"); }
return user;
});
});

```

Password Migration Script — hashpasswords.js:

```

var users = await client.query("SELECT id, name, password FROM users;");
for (var user of users.rows) {
var hashed = await bcrypt.hash(user.password, 10);
await client.query("UPDATE users SET password = $1 WHERE id = $2;",
[hashed, user.id]);
}

```

After Fix — Passwords in database stored as bcrypt hashes with salt factor 10:

```

admin | $2b$10$SiMp6v24l2qNPpXaDzEz004QHxJIKoRcz4rT1M9jPKHMnTjSe5.E2
roberto | $2b$10$J..9Bux.XBqHB0J2CC0iKeJsR8UR6sPOCvDnb05ZTR6H8RHIM/1i2

```

Even if the database is compromised, passwords cannot be reversed — bcrypt is a one-way hash with salt, making brute-force and rainbow table attacks computationally infeasible.

2.7 Fix 4 — SQL Injection Prevention (Parameterized Queries)

Vulnerability Addressed

All database queries in model/auth.js and model/products.js built SQL by directly concatenating user input — identical to the SQL Injection vulnerability confirmed in Week 1's DVWA testing with payload 1' OR '1'='1.

Before (Vulnerable) — model/products.js:

```

// SQL Injection in product search:
var q = "SELECT * FROM products WHERE name ILIKE '%" + query + "%'";

// SQL Injection in purchase:
var q = "INSERT INTO purchases(...) VALUES('" + cart.mail + "', '"
+ cart.product_name + "'..."; // All fields vulnerable

```

Fix Applied — Parameterized Queries:

```

// Auth query – parameterized:
client.query("SELECT * FROM users WHERE name = $1;", [username]);

// Search query – parameterized:

```

```

client.query("SELECT * FROM products WHERE name ILIKE $1 OR description ILIKE $1;",
['%' + query + '%']);

// Purchase query – parameterized:
client.query(
"INSERT INTO purchases(mail, product_name, user_name, product_id, " +
"address, phone, ship_date, price) VALUES($1,$2,$3,$4,$5,$6,$7,$8)",
[cart.mail, cart.product_name, cart.username, cart.product_id,
cart.address, cart.phone, cart.ship_date, cart.price]
);

```

Result: With parameterized queries, even if a user submits admin' OR '1'=1, the database treats it as a literal string value — not executable SQL. The SQL injection attack is completely neutralized across all 4 query locations in the application.

2.8 Fix 5 — JWT Token-Based Authentication

Vulnerability Addressed

The application used only server-side sessions with no cryptographically signed tokens. JWT adds a verifiable, stateless authentication layer that cannot be forged without the secret key.

Fix Applied — app.js and routes/login.js:

```

// app.js – JWT secret stored in app settings
var jwt = require('jsonwebtoken');
var JWT_SECRET = 'your-super-secret-jwt-key-2024';
app.set('JWT_SECRET', JWT_SECRET);

// routes/login.js – token generated on successful login
auth(user, password)
.then(function(data) {
req.session.logged = true;
req.session.user_name = user;
var token = jwt.sign(
{ id: data.id, username: user },
req.app.get('JWT_SECRET'),
{ expiresIn: '1h' }
);
req.session.token = token;
res.redirect(returnurl);
});

```

JWT Feature	Implementation
Algorithm	HS256 (HMAC SHA-256)
Payload	user.id and username
Expiry	1 hour — limiting session hijacking window
Storage	Stored in session for API authorization

2.9 Before vs After Comparison

Vulnerability (Week 1)	Risk	Fix Applied (Week 2)	Result
Missing CSP / Security Headers	MEDIUM	Helmet.js middleware added to app.js	Fixed
No Input Validation (XSS risk)	HIGH	validator library — isEmpty, isLength, escape	Fixed
Plaintext Password Storage	HIGH	bcrypt hashing with salt factor 10	Fixed
SQL Injection (all queries)	HIGH	Parameterized queries (\$1, \$2 placeholders)	Fixed
Weak Authentication (no tokens)	MEDIUM	JWT generation on login (1hr expiry)	Fixed
Missing Anti-Clickjacking Header	MEDIUM	Helmet sets X-Frame-Options: SAMEORIGIN	Fixed
X-Content-Type-Options Missing	LOW	Helmet sets X-Content-Type-Options: nosniff	Fixed

2.10 Files Modified & Packages Installed

Files Modified:

File	Changes Made
app.js	Added Helmet middleware, JWT secret configuration
routes/login.js	Added validator library, input validation checks, JWT token generation
model/auth.js	Replaced pg-promise with pg, added bcrypt comparison, parameterized SQL query
model/products.js	Replaced pg-promise with pg, all 4 queries converted to parameterized form
hashpasswords.js	One-time script to migrate plaintext passwords to bcrypt hashes in DB

File	Changes Made
bin/www	HTTPS server configuration added (Week 3 update)

Packages Installed:

Package	Version	Purpose
helmet	4.x	Sets secure HTTP response headers
validator	latest	Input validation and sanitization
bcrypt	latest	Password hashing with salt
jsonwebtoken	latest	JWT token generation and verification
pg	8.x	PostgreSQL client (replaced pg-promise for Node 18 compatibility)

2.11 Conclusion

All five security fixes outlined in the Week 2 task were successfully implemented and verified on a locally running Node.js/Express application. The vulnerabilities addressed directly correspond to the critical findings from the Week 1 DVWA assessment — Reflected/Stored XSS, SQL Injection, weak password storage, missing security headers, and weak authentication. The most impactful improvements were the transition from plaintext passwords to bcrypt hashing, the elimination of SQL injection through parameterized queries, and the addition of Helmet.js which resolved seven of the header-related vulnerabilities identified by OWASP ZAP in Week 1 in a single step. These fixes demonstrate the practical application of defense-in-depth principles: layering multiple security controls so that even if one layer is bypassed, others remain in place to protect the application.

WEEK 3 — ADVANCED SECURITY & FINAL REPORTING

Application: vulnerable-node (NodeVulnerable) | Tools: Nmap, Winston, OpenSSL | Date: June 12, 2026

Prepared By	Ayesha Sarwar Khan
Organization	Developers Hub Corporation
Date	June 12, 2026
Application Tested	vulnerable-node (NodeVulnerable) — Node.js/Express
Assessment Type	Penetration Testing, Logging & HTTPS Implementation
Tools Used	Nmap, Winston, OpenSSL

3.1 Executive Summary

This report documents the work completed during Week 3 of the Cybersecurity Internship at Developers Hub Corporation. Building on the security fixes implemented in Week 2 on the vulnerable-node application, Week 3 focused on three areas: basic penetration testing using Nmap to verify the effectiveness of previous fixes, implementation of centralized security logging using the Winston library, and the deployment of HTTPS via a self-signed SSL certificate to secure data transmission. All three tasks were completed and verified successfully.

Task	Status	Tool/Library Used
Basic Penetration Testing	Completed	Nmap
Basic Logging Setup	Completed	Winston
HTTPS Implementation	Completed	OpenSSL (self-signed cert)
Security Checklist	Completed	—

3.2 Basic Penetration Testing — Nmap

Nmap was used to perform basic penetration testing against the application running on localhost (127.0.0.1). Three separate scans were performed to assess open ports, service versions, response headers, and known vulnerabilities.

3.2.1 Service & Version Detection Scan

```
Command: nmap -sV 127.0.0.1
```

This scan identified all open ports on the local machine and attempted to detect the service and version running on each. Port 3000/tcp was confirmed open, running the Node.js/Express application. Other ports (135, 445, 1001, 2179, 5357, 5432) correspond to standard Windows services and the local PostgreSQL database.

3.2.2 Targeted Port Scan with Header Fingerprinting

```
Command: nmap -sV -p 3000 127.0.0.1
```

This scan targeted port 3000 specifically and performed a deeper service fingerprint. Nmap captured the raw HTTP response headers returned by the application, which confirmed that the security headers configured via Helmet.js in Week 2 are active and being sent with every response.

Header	Value Observed
Content-Security-Policy	default-src 'self'; object-src 'none'; script-src 'self'...
X-Frame-Options	SAMEORIGIN
Strict-Transport-Security	max-age=15552000; includeSubDomains
X-Content-Type-Options	nosniff
X-XSS-Protection	0 (modern CSP-based protection active)
Referrer-Policy	no-referrer
X-DNS-Prefetch-Control	off
X-Download-Options	noopen
X-Permitted-Cross-Domain-Policies	none

Significance: The presence of these headers in the raw Nmap output is direct external evidence that the Helmet.js middleware added in Week 2 is functioning correctly in a running instance of the application — verified independently of the application's own code or browser DevTools.

3.2.3 Vulnerability Script Scan

```
Command: nmap --script vuln -p 3000 127.0.0.1
```

The Nmap vulnerability scanning script suite (--script vuln) was run against port 3000 to check for known, commonly exploitable vulnerabilities (e.g. Slowloris DoS, common CVEs for web servers). The scan completed in approximately 10.56 seconds and returned no vulnerability findings for the service on port 3000.

3.2.4 Penetration Testing Summary

Scan Type	Command	Result
Service/Version Detection	<code>nmap -sV 127.0.0.1</code>	Port 3000 confirmed open, Node.js/Express identified
Targeted Header Scan	<code>nmap -sV -p 3000 127.0.0.1</code>	All Week 2 Helmet.js security headers confirmed active
Vulnerability Scan	<code>nmap --script vuln -p 3000 127.0.0.1</code>	No critical vulnerabilities found on application port

3.3 Basic Logging Implementation — Winston

To improve visibility into application behavior and support future security monitoring, the Winston logging library was integrated into the application alongside the existing log4js logger.

3.3.1 Implementation

```
npm install winston
```

Configuration added to app.js (renamed to securityLogger to avoid naming conflict with the existing Morgan logger variable named logger):

```
const winston = require('winston');
const securityLogger = winston.createLogger({
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'security.log' })
  ]
});
securityLogger.info('Application started');
```

3.3.2 Issue Encountered & Resolution

Attribute	Detail
Issue	SyntaxError: Identifier 'logger' has already been declared
Root Cause	Morgan HTTP logger was already imported as 'var logger' on line 7 of app.js
Resolution	Renamed Winston instance from 'logger' to 'securityLogger' to avoid collision

3.3.3 Verification

Upon restarting the application with `npm start`, the console immediately output a structured JSON log entry, confirming Winston was active:

```
{"level":"info","message":"Application started"}
```

A new file, security.log, was created in the project root directory containing the identical log entry, confirming the File transport is also working correctly. The existing log4js logger continued to function normally, demonstrating that both logging systems can coexist.

3.4 Secure Data Transmission — HTTPS

To address the 'Use HTTPS for data transmission' checklist item, a self-signed SSL/TLS certificate was generated and an HTTPS server was added to the application alongside the existing HTTP server.

3.4.1 Certificate Generation

Using OpenSSL (via Git Bash), a self-signed certificate and private key were generated:

```
openssl req -nodes -new -x509 -keyout server.key -out server.cert -days 365
```

Field	Value
Country (C)	PK
State/Province (ST)	Punjab
Locality (L)	Gujranwala
Organization (O)	Developers Hub Corporation
Organizational Unit (OU)	Cybersecurity Internship
Common Name (CN)	localhost
Validity	365 days
Output Files	server.key (private key), server.cert (public certificate)

3.4.2 Server Configuration — bin/www

```
var https = require('https');
var fs = require('fs');
var httpsOptions = {
  key: fs.readFileSync('./server.key'),
  cert: fs.readFileSync('./server.cert')
};
var httpsServer = https.createServer(httpsOptions, app);
httpsServer.listen(3443);
console.log('HTTPS Server running on https://localhost:3443');
```

3.4.3 Verification

Navigating to <https://localhost:3443> successfully loaded the application's login page over an encrypted HTTPS connection. The browser displayed a 'Not Secure' indicator, which is expected and normal behavior for self-signed certificates that are not issued by a publicly trusted Certificate Authority. The encrypted connection itself was established successfully.

***Production Note:** In a production environment, a certificate from a trusted CA (e.g. Let's Encrypt) would be used instead of a self-signed certificate to avoid browser security warnings, while maintaining the same underlying encryption mechanism.*

3.5 Final Security Best Practices Checklist

The following checklist summarizes all security best practices implemented across all three weeks of this internship on the vulnerable-node application:

#	Best Practice	Status	Implementation	Week
1	Validate all inputs	DONE	validator library	Week 2
2	Use HTTPS for data transmission	DONE	Self-signed SSL certificate, port 3443	Week 3
3	Hash and salt passwords	DONE	bcrypt, salt factor 10	Week 2
4	Secure HTTP response headers	DONE	Helmet.js middleware	Week 2
5	SQL Injection prevention	DONE	Parameterized queries (\$1, \$2)	Week 2
6	Logging and monitoring	DONE	Winston + log4js	Week 3
7	Token-based authentication	DONE	jsonwebtoken, 1hr expiry	Week 2
8	Vulnerability scanning	DONE	OWASP ZAP (Week 1), Nmap (Week 3)	Wk 1 & 3

3.6 Final Conclusion

Week 3 successfully completed the security lifecycle established over the internship: Week 1 identified vulnerabilities through assessment and scanning, Week 2 implemented code-level fixes for those vulnerabilities, and Week 3 verified those fixes through independent penetration testing (Nmap), added operational visibility through structured logging (Winston), and secured the data transmission layer with HTTPS.

The Nmap scans provided independent confirmation — separate from the application's own code or browser tools — that the Helmet.js security headers from Week 2 are correctly applied in the running application, and that no critical vulnerabilities are exposed on the application's port. The

addition of Winston logging and HTTPS encryption completes the defense-in-depth approach: vulnerabilities were found, fixed, verified, and the application is now monitored and transmits data securely.

All checklist items from across all three weeks have been completed and verified, marking the successful conclusion of the three-week security assessment and remediation project on the vulnerable-node application.

Prepared By: Ayesha Sarwar Khan	Organization: Developers Hub Corporation	Submission Date: June 26, 2026
--	---	---------------------------------------

CONFIDENTIAL — FOR INTERNSHIP ASSESSMENT PURPOSES ONLY